

# HASHING

Dr. Öğr. Üyesi Fatma AKALIN

**Liste yapısında** (bağlantılı liste gibi) ekleme, arama ve silme işlemleri genellikle  **$O(N)$**  zaman alır, çünkü listeyi baştan sona taramak gerekebilir.

**İkili arama ağacında** (binary search tree) ekleme, arama ve silme işlemleri  **$O(\log N)$**  zaman karmaşıklığına sahiptir, çünkü her işlemde ağacın yüksekliğiyle orantılıdır.

**Dizi veri yapısında** (array), bir elemana erişim  **$O(1)$**  zaman alır (indeks için), çünkü indeksleme doğrudan yapılabilir. Ancak, ekleme ve silme işlemleri, dizinin boyutuna bağlı olarak  **$O(N)$**  olabilir.

**Arama işlemleri** içinde bir **adres polinomu** ya da **fonksiyonu** kullanılabilmesi halinde **ekleme silme ve arama** işlemleri bir **çırpıda** yapılabilir. Bu temel işlemleri **sabit zamanda bir çırpıda** gerçekleştirebilmek için **hash(çırpı) tablosu** yapısı geliştirilmiştir.

# HASHING (ÇIRPILAMA)

Özetleme (hash) fonksiyonları, değişken uzunluktaki girdileri alarak sabit uzunlukta ve daha kısa temsillere dönüştüren matematiksel fonksiyonlardır. Bu fonksiyonlar, veri bütünlüğünün doğrulanması (örneğin, verinin değiştirilip değiştirilmediğinin tespit edilmesi) ve veri sınıflandırma süreçlerinde etkin bir biçimde kullanılmaktadır.

AKALIN

Anlaşılması en basit özetleme fonksiyonu **modül** işlemidir. Buna göre örneğin mod 10 işlemini ele alalım, aşağıdaki sayıların mod 10 sonuçları listelenmiş ve gruplanmıştır:

Dr. ...

| Demet (Buket, Bucket) | Sayılar |      |     |    |    |    |
|-----------------------|---------|------|-----|----|----|----|
| 0                     |         |      |     |    |    |    |
| 1                     | 81      |      |     |    |    |    |
| 2                     | 12      | 432  | 62  |    |    |    |
| 3                     | 3       |      |     |    |    |    |
| 4                     | 4       | 34   | 344 | 54 | 94 | 44 |
| 5                     | 95      |      |     |    |    |    |
| 6                     |         |      |     |    |    |    |
| 7                     | 17      | 67   |     |    |    |    |
| 8                     | 8       |      |     |    |    |    |
| 9                     | 389     | 2339 | 349 | 9  |    |    |

Kısaca yandaki sayıların hepsi 1 haneli bir sayıya özetlenmiştir. Örneğin 81 -> 1, 344 -> 4 gibi. Elbette aynı sayıya **özetlenen birden fazla sayı** bulunmaktadır. Bu duruma **çakışma (collusion)** adı verilmektedir.

Somutlaştırmak için verdiğimiz bu örnek üzerinden hash fonksiyonlarını ayrıntıları ile irdeleyelim.

Bir **hash** tablosu yapısı **sabit boyutludur** ve **anahtarlardan** oluşmaktadır. **Veriye** bir **anahtar(key)** yardımı ile **erişilecek** biçimde basit bir **dizi** yapısında tasarlanmaktadır. Her bir **anahtar**, bir **değere** karşılık gelir ve bu **katar(string)** ya da **sayı** olabilir. **Anahtarlar** bir **indeks üretmek** için kullanılır ve **dizideki elemanlara** bu **indekslerle** ulaşılır. Buradaki **anahtarlar tekildir**, bir başka **kayıtta aynı anahtar** **olamaz**. Ancak **veri aynı** olabilir. Aynı **ad ve soyadına** sahip olabilmelerine rağmen insanların **kimlik numaraları** kesinlikle **farklıdır**. Dolayısıyla burada kişilerin kimlik numarası **anahtardır**. Kişinin adı soyadı ise bu **anahtarı** ait veridir.

Herhangi bir anahtarın tablodaki **hangi indeksi tanımladığı bir fonksiyonla** belirlenir. Bu tanıma gerçekleştiren **fonksiyona Hash fonksiyonu** denir. Bir hash fonksiyonu kaydedilmek üzere girilen bir **veriyi alır ve bellekte onu yerleştirmek üzere ilk seçeneği** belirler.

Bir hash fonksiyonu **basit hesaplanabilmeli ve birbirinden farklı anahtar değerlerini farklı indislere yerleştirmelidir** Ancak her zaman böyle bir hash fonksiyonu bulunmayabilir. Uygulamada mümkün olduğunca her **farklı anahtarı farklı indise yerleştirilecek ideal bir hash fonksiyonu** bulunmaya çalışılır.

Elemanların tam sayı olması halinde hash fonksiyonunu elde etmek kolaydır. Elemanları sayı olan bir tablo için hash fonksiyonu yandaki gibi tanımlanabilir.

```
int HashFonk(string anahtar, int
uzunluk){
    int hashDeger=0;
    for(int i=0; i<anahtar.length(); i++){
        hashDeger += anahtar[i];
    }
    return hashDeger % uzunluk;
}
```

Diyelim ki anahtar = "abc" ve uzunluk = 10 olsun:

'a' = 97

'b' = 98

'c' = 99

**Toplam:**  $97 + 98 + 99 = 294$

**Sonuç:**  $294 \bmod \{10\} = 4$  (Bu anahtar, tablonun 4. indeksine yerleştirilir).

$$\text{hash}(x) = X \bmod (\text{TabloMaxElemanSayısı})$$

Örneğin, 11 elemana sahip bir tabloda 15,558,32,132,102 ve 5 anahtarlarını tabloya  $h(x) = x \bmod 11$  olarak belirlenen hash fonksiyonu ile yerleştirmek istersek belirlenen yerleşim aşağıdaki gibi olacaktır.

|     |   |   |     |    |   |   |   |     |   |    |
|-----|---|---|-----|----|---|---|---|-----|---|----|
| 132 |   |   | 102 | 15 | 5 |   |   | 558 |   | 32 |
| 0   | 1 | 2 | 3   | 4  | 5 | 6 | 7 | 8   | 9 | 10 |

NOT: Basit bir fonksiyon olmasının yanında eğer **performans isteniyorsa daha karmaşık ölçümler** fonksiyona eklenmelidir



Burada tablo eleman sayısının **11** olması tablonun oluşmasında kolaylık sağlamıştır. Eğer tablo eleman sayısı **10** seçilmiş olsaydı muhtemelen **aynı anahtara karşılık gelen indis çok** olacaktı. Çünkü **21,31, 41..** elemanların hepsi mod 10'a göre **aynı değeri** verir. Bu durumun sonucunda da birçok sayı tablonun aynı satırına denk gelir ve böylece **çakışma** adı verilen bir durumla karşılaşılır. Bu yüzden tablo eleman sayısının **asal sayı** seçilmesi daha uygundur.

Unutulmamalıdır ki çakışmalar **hash uygulamalarında istenmeyen bir durumdur ve giderilmelidir**. Bunun için açık çarpılama, kapalı çarpılama veya ikili hashing (Double Hashing) gibi yöntemler kullanılmaktadır

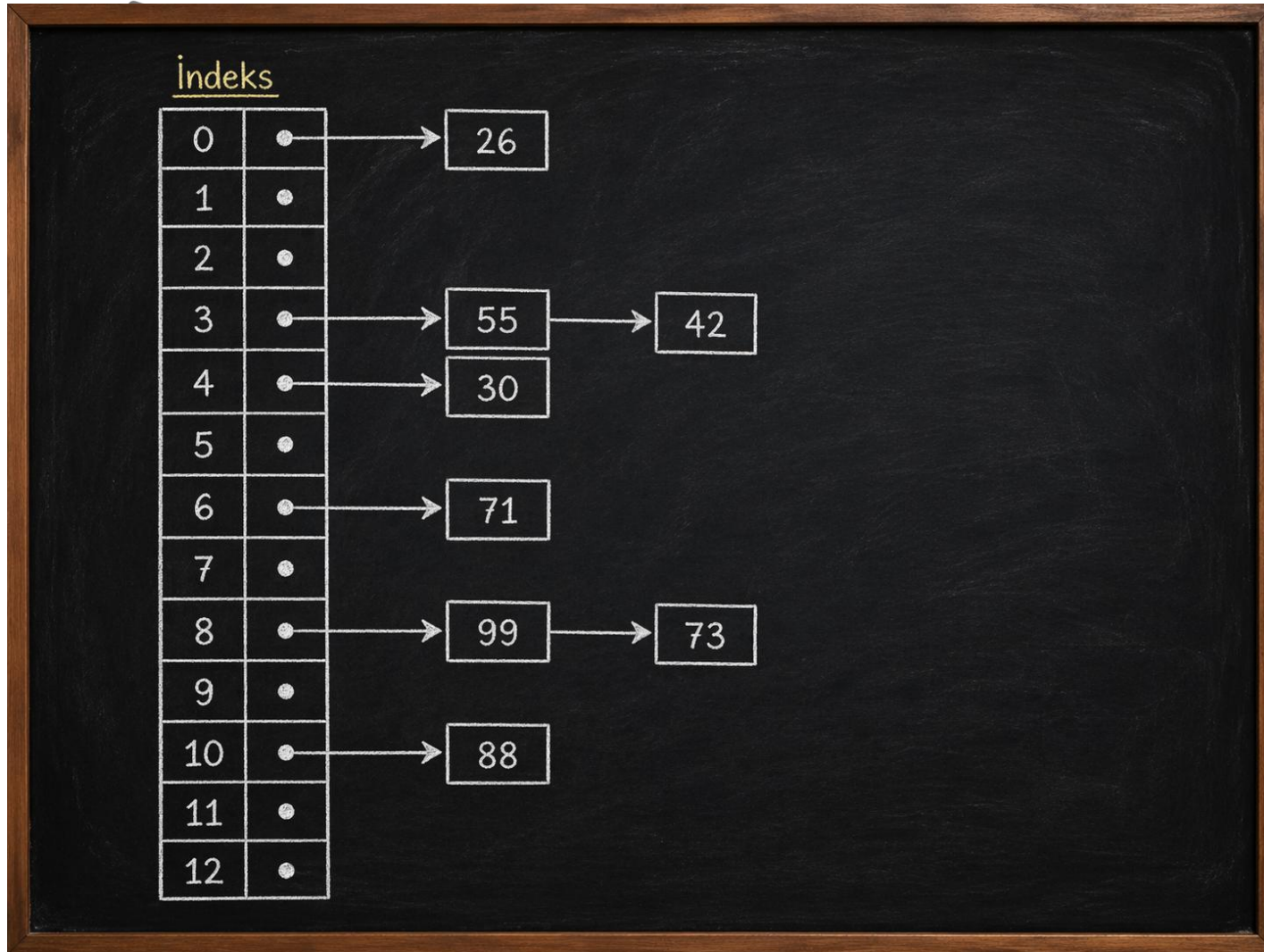
# Açık Çırpılama (Ayrık Zincirleme)

Bu yöntemde **aynı değere hash edilen elemanların bir listesi** tutulur. **Temelde** tablodaki her bir bölümü liste şeklinde tanıtarak **bağlı liste** kullanılmaktadır. Bir **veri eklenmek** istendiğinde **hash fonksiyonu** bu veri için **daha önceki bir verinin indeksine üretiyorsa** yani **çakışma** oluyorsa tablodaki **her indeksin listenin başı** olduğunu varsayarak aynı özellikteki elemanlar **aynı listeye eklenir**. Her **eleman listenin başına** eklenirse **ekleme maliyeti sizce ne olacaktır?**

Aşağıdaki dataları ayırık zincirleme kullanarak hash tabloya yerleştirelim.

Veri: 99 88 30 55 26 42 71 73

Index: 8 10 4 3 0 3 6 8



Örnek 1

ma AKALIN

Yöntemin çalışması inceleyelim. Bir eleman aranmak istendiğinde önce **indeks** bulunacaktır. **Index bulunduktan sonra** bu andan itibaren bir **bağlı liste** ile ilgiliyiz ve arayacağımız elemana **bağlı listenin arama fonksiyonu** ile bulunanlar söz konusudur. Bu ise **yine** bir arama olacaktır. Böylece **genel arama zaman maliyetimiz bu noktadan itibaren sabit zaman maliyetinden uzaklaşmaya başlayacaktır. En kötü** durumda bütün elemanların **indeks değeri aynı** ise elemanların hepsi **aynı noktada birikecek** ve **n** elemanlı **bağlı bir listede** arama yapıyormuş gibi olunacak ve zaman maliyetimiz  $O(n)$  olacaktır.

Bağlı liste (chaining) yöntemi kullanılarak gerçekleştirilen çakışma çözümleme yaklaşımlarında, her bir tablo hücresinin bir işaretçi (pointer) aracılığıyla dinamik olarak oluşturulan bir listeyi göstermesi, uygulama (implementasyon) karmaşıklığını artırmaktadır. Bu durum, özellikle bellek yönetimi açısından önem taşımakta olup, malloc ve free gibi dinamik bellek ayırma ve serbest bırakma işlemlerinin teorik olarak sabit zamanlı kabul edilmesine rağmen, pratikte ek yük (overhead) oluşturarak çalışma zamanını olumsuz etkileyebilmektedir.

Hash tablosunda bir arama işleminin zaman maliyeti genel olarak iki bileşenden oluşmaktadır: (i) çırpı (hash) fonksiyonunun hesaplanması için gereken sabit süre ve (ii) ilgili indeks altında bulunan bağlı listede ilerleme süresi. Bu nedenle, çakışmaların yoğun olduğu durumlarda liste uzunluklarının artması, arama işleminin ortalama zaman karmaşıklığını doğrudan artırmaktadır.

Bu tür olumsuzlukların önüne geçebilmek için, hash tablosunun boyutunun uygun şekilde seçilmesi kritik bir öneme sahiptir. Özellikle, anahtarların tabloya daha dengeli (uniform) bir biçimde dağılmasını sağlamak amacıyla tablo boyutunun asal (prime) bir sayı olarak belirlenmesi yaygın bir yaklaşımdır. Asal sayı seçimi, belirli aritmetik düzenliliklerin (patterns) etkisini azaltarak çakışma olasılığını düşürmekte ve böylece tablo içerisinde yığılmaların (clustering) önüne geçilmesine katkı sağlamaktadır. Bu sayede, hem arama hem de ekleme işlemlerinin ortalama performansı iyileştirilmektedir.



# Kapalı Çırpılama (Açık Adresleme)

Kapalı çırpılama yönteminde bir **çakışma olduğu zaman boş bir hücre bulunana kadar başka hücreler** denenir. Hash tablosunun her satırında bir eleman vardır ya da bu satır boştur. **Bir eleman aranacağı zaman** belirlenen bir fonksiyonla **sınamalar** yapılır ve **boş bir satıra** rastlandığında **çıkılır**. Böyle bir elemanın olmadığına karar verilir. **Ayrık çırpılamadan farklı** olarak elemanlar yerleştirilirken bir **listeye zincir biçiminde değil de fonksiyona göre boş bir yere buluncaya** kadar aranır ve boş bir yeri bulunca buraya yerleştirilir. Böylece **ayrık çırpılamada oluşan dezavantajların da önüne geçilmiş** olunur. Çünkü burada **liste veri yapısı gibi yeni yapı oluşturulmamaktadır**.

# A-Doğrusal Sınama Yöntemi

Hash fonksiyonu ile tabloyu bir değer eklenirken **çakışma** oluşuyorsa eleman tabloda **birer hücre aşağı inerek** bulunan **ilk boş hücreye** yerleştirilir.

**Örnek 2:** Tablomuzda daha öncesinden 26, 88, 30, 70, 99, 73, 62, 43, 18 elemanlarının bir hash fonksiyonu ile tabloya nasıl yerleştirildiğini görelim. Başlangıçta tablodan 26, 71, 99, 73, 88 değerleri bulunmaktadır. 30 elemanını eklemek istiyoruz. Şimdi süreci nasıl yürüteceğimizi inceleyelim.



30 elemanı eklemek istedik ( $30\%13=4$ ) ve fonksiyon bize 4 indeksini üretti. Tabloda 4 indisi ile gösterilen yer boştur ve **direkt** olarak **yerleştirdik**. 75 verisi için fonksiyon 10 indisini üretti. Tabloda 10 indisiyle gösterilen adreste 88 elemanı bulunmaktadır. Bu durumda çarpışma oluştu ve  $F(i)=1$  değerini 10 indisine ekledik. Bu durumda yeni veriyi 11 indisinin gösterdiği adrese ekledik. 62 elemanını eklerken 2 defa çarpışma oluştu. 62 için 10 indisi oluştu. 10 indisi ile gösterilen yerde 88 var. Boş olmadığından çarpışma oluştu. Bu yüzden indisi bir arttırdık ve 11 oldu. 11 ile gösterilen adreste ise 75 elemanı var. Yine çarpışma oluştu. Tekrar bir arttırdık ve 12 ile gösterilen yer boş olduğundan 62 verisini buraya yerleştirdik

| 30 Ekle | 75 Ekle | 62 Ekle | 18 Ekle | 43 Ekle |
|---------|---------|---------|---------|---------|
| 0       | 0       | 0       | 0       | 0       |
| 1       | 1       | 1       | 1       | 1       |
| 2       | 2       | 2       | 2       | 2       |
| 3       | 3       | 3       | 3       | 3       |
| 4       | 4       | 4       | 4       | 4       |
| 5       | 5       | 5       | 5       | 5       |
| 6       | 6       | 6       | 6       | 6       |
| 7       | 7       | 7       | 7       | 7       |
| 8       | 8       | 8       | 8       | 8       |
| 9       | 9       | 9       | 9       | 9       |
| 10      | 10      | 10      | 10      | 10      |
| 11      | 11      | 11      | 11      | 11      |
| 12      | 12      | 12      | 12      | 12      |

|    |    |    |    |    |
|----|----|----|----|----|
| 26 | 26 | 26 | 26 | 26 |
|    |    |    |    |    |
|    |    |    |    |    |
|    |    |    |    |    |
| 30 | 30 | 30 | 30 | 30 |
|    |    |    | 18 | 18 |
| 71 | 71 | 71 | 71 | 71 |
|    |    |    |    | 43 |
| 99 | 99 | 99 | 99 | 99 |
| 73 | 73 | 73 | 73 | 73 |
| 88 | 88 | 88 | 88 | 88 |
|    | 75 | 75 | 75 | 75 |
|    |    | 62 | 62 | 62 |

Eğer tablonun **eleman kapasitesi yeterli düzeyde** ise eleman **eklemek** çok **zor** olmayacaktır. Ama **tablo dolmaya başladıkça** algoritmanın çalışma süresi de artacaktır.

**Doğrusal sinama** basit olması ve bütün satırları sinaması nedeniyle tercih edilebilir. Ancak **sıralı bakılması** nedeniyle zamanla dolu satırlar birleşerek **büyük zincirler** oluşturmakta ve boş satır bulma olasılığı azalmaktadır.

Çakışmaların (collision) azaltılmasında en temel yaklaşımlardan biri, hash tablosunun boyutunun artırılmasıdır. Çakışma olasılığı, büyük ölçüde tablonun doluluk oranı (load factor) ile ilişkilidir; tablo ne kadar doluyorsa, farklı anahtarların aynı indekse düşme ihtimali o kadar artmaktadır. Bu nedenle, doluluk oranının belirli bir eşik değeri aşması durumunda tablo boyutunun artırılması, performansın korunması açısından kritik bir öneme sahiptir.

# İkili Çırpılama (Double Hashing)

Bu yöntemde **iki kere hash işlemi** yapılır. Bunun amacı **belli indekslerdeki kümelemenin önüne geçmektir**. İlk hash yapıp yerleşeceği indeks değeri belli olduktan sonra eğer o index değeri dolu değilse ikinci hash işlemi direkt elenir. **Dolu ise ikinci hash değeri hesaplanır**. İkinci hash değeri ne kadar ileri öteleneneceğini gösterir

## Örnek 3

$\text{Hash1}(\text{değer}) = \text{değer} \% \text{DiziBoyutu}$

$\text{Hash2}(\text{değer}) = (\text{indeks} + i * \text{adimsayisi}) \% \text{DiziBoyutu}$

Dr. Öğr. Üyesi Fatma AKALIN

Hash1(değer)=değer%DiziBoyutu

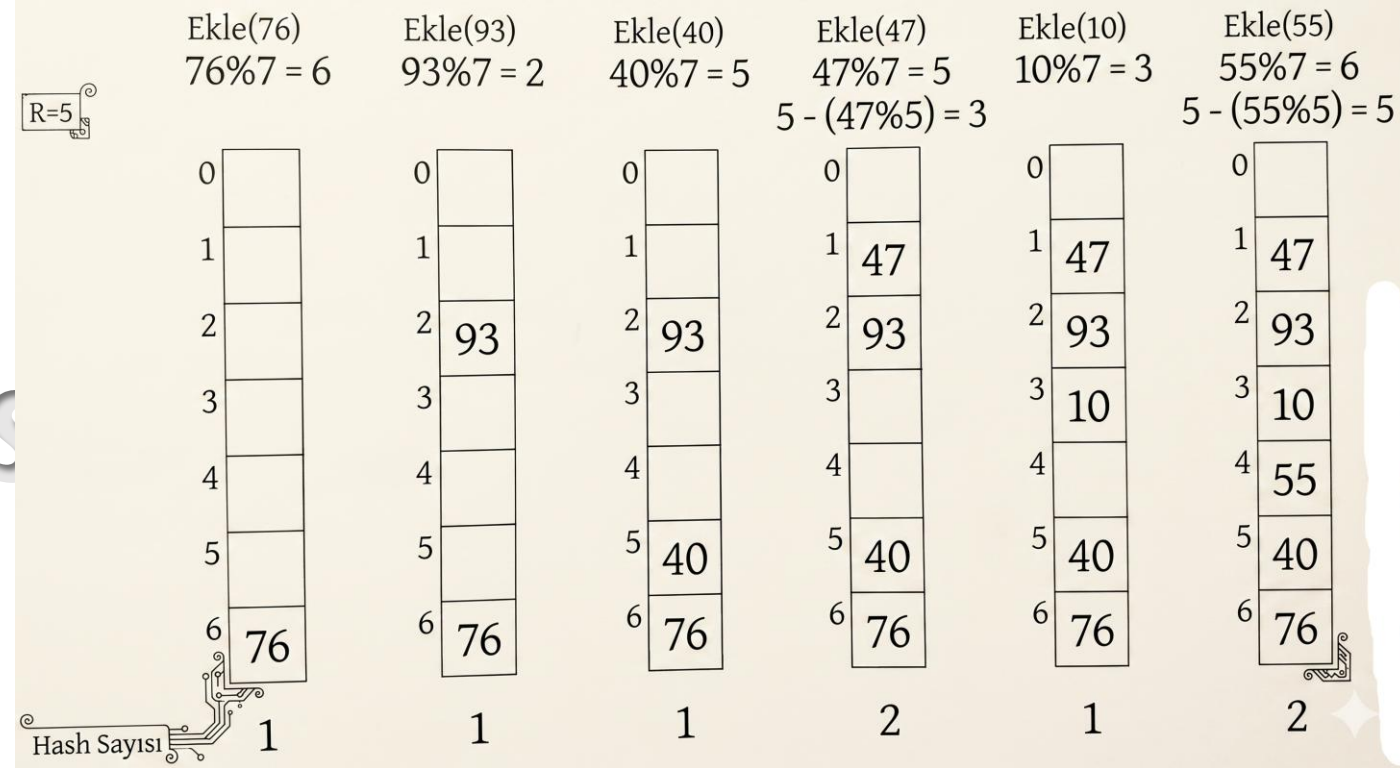
Hash2(değer)=(indeks+i\*adimsayisi)% DiziBoyutu

Yandaki örnekte **7 elemanlı bir dizi hash tablosu** olarak kullanılacaktır. 76 sayısı eklendiğinde 7 ile bölümünden kalan 6 olarak nitelendirilecektir. Dolayısıyla ilgili sayı 6 indeksine yerleşecektir. **6. index boş olduğu için 2. hash fonksiyonunu hesaplamaya gerek yoktur.**

93 sayısı da aynı şekilde bir hash fonksiyonu ile eklenir. Fakat 47 sayısı 40 ile aynı indekse sahip olduğu için çarpışma gerçekleşecektir ve **ikinci hash fonksiyonu hesaplanacaktır.** 7 sayısını en yakın asal sayı 5 olduğu için  $R=5$  seçilir. 47 sayısı için de ikinci hash fonksiyonu **3 değerini döndürecek**ti. Bu değer normal olması gereken indeksten kaç adım öteleneneceğini gösterir. Dolayısıyla 1 numaralı indeks'e yerleşir.

Hash2(değer)=(indeks+i\*adimsayisi)% DiziBoyutu

$$\text{adres}=(5+1\times 3)\text{mod}7=8\text{mod}7=1$$



Hash2(değer)=(indeks+i\*adimsayisi)% DiziBoyutu

$$\text{adres}=(6+1\times 5)\text{mod}7=11\text{mod}7=4$$

NOT : İkinci hash "**müsait bir yer bulma**" operasyonudur. İkinci de yine çarpışma olursa 3. ya da 4. yer bulmada da ikinci hash çalışacaktır. Sonraki sayfalardaki kodlama işlemi bu mantık üzerinde yazılmıştır.



Sizler tarafından gerçekleştirilecek açık çarpılama işlemi için gerekli fonksiyonlar yanda verilmiştir.

# AÇIK ÇIRPILAMANIN C++ İLE GERÇEKLEŞTİRİMİ

```
#ifndef HASH_HPP
#define HASH_HPP

#include <iostream>
using namespace std;
#define MAX 101

struct Dugum{
    int veri;
    Dugum *ileri;
    Dugum(int vr,Dugum *ilr=NULL){
        veri=vr;
        ileri=ilr;
    }
};

class Hash{
private:
    Dugum* Dizi[MAX];
public:
    Hash();
    int HashCode(int);
    void Ekle(int);

    bool Bul(int);

    void Yaz() const;
    ~Hash();
};

#endif
```

```
#include "Hash.hpp"

Hash::Hash(){
    for(int i=0;i<MAX;i++){
        Dizi[i] = NULL;
    }
}

void Hash::Ekle(int sayi){
    if(Bul(sayi)) return;
    int indeks = HashCode(sayi);
    Dizi[indeks] = new Dugum(sayi,Dizi[indeks]);
}

bool Hash::Bul(int sayi) {
    int indeks = HashCode(sayi);
    Dugum *tmp = Dizi[indeks];
    while(tmp != NULL){
        if(tmp->veri == sayi) return true;
        tmp=tmp->ileri;
    }
    return false;
}

int Hash::HashCode(int sayi){
    return sayi%MAX;
}
```

```
void Hash::Yaz() const{
    for(int i=0;i<MAX;i++){
        if(Dizi[i] != NULL){
            cout<<"Dizi["<<i<<"]=";
            for(Dugum *tmp = Dizi[i];tmp != NULL;tmp=tmp->ileri){
                cout<<tmp->veri<<"->";
            }
            cout<<"NULL"<<endl;
        }
    }
}

int main() {
    Hash *hash = new Hash();
    hash->Ekle(100);
    hash->Ekle(55);
    hash->Ekle(1235);
    hash->Ekle(10000);
    hash->Ekle(1);
    hash->Ekle(102);
    //hash->Ekle(102);
    //hash->Ekle(102);
    hash->Yaz();
    delete hash;

    return 0;
}
```

ALIN

Kod örneği c/c++ ile veri yapıları ve çözümlü uygulamalar kitabından alınmıştır. Nejat yumuşak, Fatih Adak

# DOUBLE HASHING'IN C++ İLE GERÇEKLEŞTİRİMİ

Sizler tarafından gerçekleştirilecek açık  
çırpılama işlemi için gerekli fonksiyonlar yanda  
verilmiştir.

```
#ifndef DOUBLEHASHING_HPP
#define DOUBLEHASHING_HPP

#include <iostream>
using namespace std;

#define MAX 101
#define YAKINASAL 97

#include "ElemanYok.hpp"
#include "TabloDolu.hpp"

// Eksi değer kabul edilmeyen Double Hashing
class DoubleHashing{
private:
    int *tablo;
    int elemanSayisi;

public:
    DoubleHashing();
    int Hash1(int);
    int Hash2(int);
    bool Dolumu() const;
    void Ekle(int);

    void Yaz();
    ~DoubleHashing();
};
#endif
```

```
void DoubleHashing::Yaz(){
    for (int i = 0; i < MAX; i++)
    {
        if (tablo[i] != -1)
            cout<<"["<<i<<" ]\t:"<<tablo[i]<<endl;
        else
            cout<<"["<<i<<" ]\t:"<<endl;
    }
}

DoubleHashing::~DoubleHashing(){
    delete [] tablo;
}

int main(){
    int dizi[] = {15,9,62,34,59,37,85,42,102,1,135};
    DoubleHashing *tablo = new DoubleHashing();
    for(int i=0;i<11;i++) tablo->Ekle(dizi[i]);
    tablo->Yaz();
    delete tablo;
}
```

```
#include "DoubleHashing.hpp"

DoubleHashing::DoubleHashing(){
    tablo = new int[MAX];
}

int DoubleHashing::Hash1(int anahtar){
    return anahtar % MAX;
}

int DoubleHashing::Hash2(int anahtar){
    return YAKINASAL - (anahtar % YAKINASAL);
}

bool DoubleHashing::Dolumu() const{
    return elemanSayisi == MAX;
}

void DoubleHashing::Ekle(int anahtar){
    if(Dolumu()) throw TabloDolu();

    int indeks = Hash1(anahtar);
    if(tablo[indeks] != -1){ // Çarpışma Olur
        int adimSayisi = Hash2(anahtar);
        int i=1;
        while(true){
            int yeniIndeks = (indeks + i * adimSayisi) % MAX;
            if(tablo[yeniIndeks] == -1){ // Çarpışma Yoksa
                tablo[yeniIndeks] = anahtar;
                break;
            }
            i++;
        }
    }
    else{
        tablo[indeks] = anahtar;
    }
    elemanSayisi++;
}
```

Kod örneği c/c++ ile veri yapıları ve çözümlü uygulamalar kitabından alınmıştır.  
Nejat yumuşak, Fatih Adak

# KAYNAKÇA

<https://bilgisayarkavramlari.com/2008/05/26/ozetleme-fonksiyonlari-hash-function/>

Kod örneği c/c++ ile veri yapıları ve çözümlü uygulamalar kitabından alınmıştır. Nejat yumuşak, Fatih Adak, Seçkin Yayınları. 3 .Baskı

<https://chatgpt.com/>

<https://gemini.google.com/>